



MERRIMACK COLLEGE

CSC 6303

Week 5

Operating Systems basic concepts and organization

Systems and Languages Survey - Dr. Paulo Fernandes

Presentation Agenda

Week 5

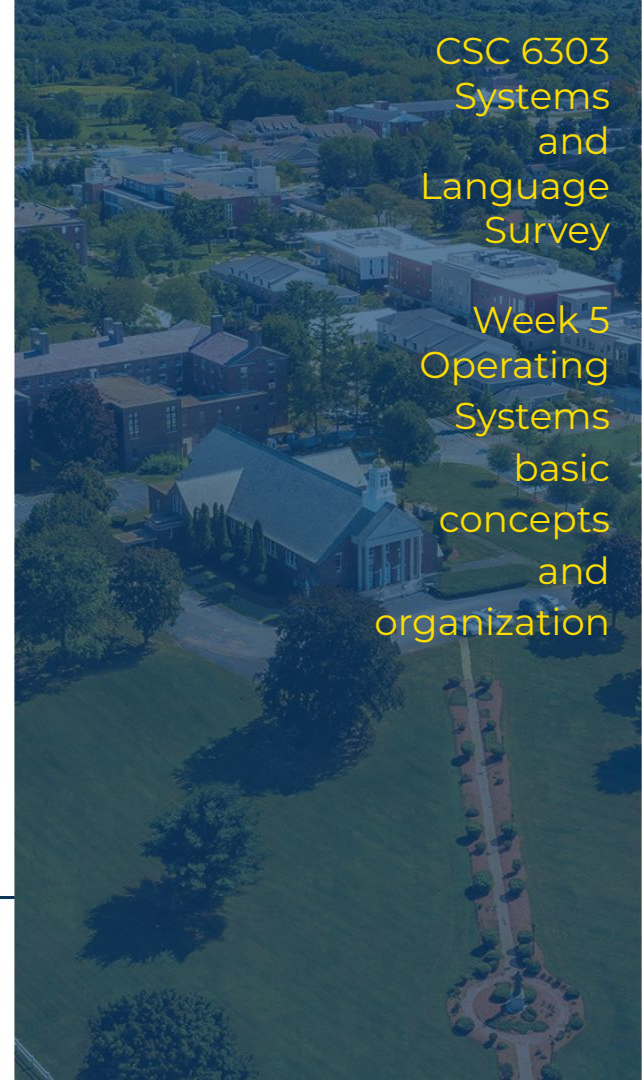
- Operating Systems
 - a. Basic Concepts
 - b. Organization
- Command Line Interface
 - a. sh scripts for Linux/macOS
 - b. sh scripts for Windows
 - c. System calls within high-level languages
- This Week's tasks



MERRIMACK COLLEGE

CSC 6303
Systems
and
Language
Survey

Week 5
Operating
Systems
basic
concepts
and
organization



Operating Systems

Is a device defined by the hardware or by the software?

The first contact we have with a computer (or any other device) is through its Operating System (OS). It has been like that for the first computers back in the 50's and it is still in the latest wearable device you may be wearing, your cell phone, or your tv set.

In all these, the OS is the basic program that is running all the time and we will see it through its:

- Basic Concepts
- Organization

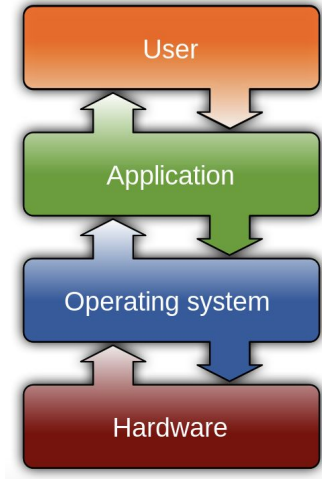


OS History

"An operating system (OS) is system software that manages computer hardware, software resources, and provides common services for computer programs."

The first computers had only a loading program (resident monitor) that was intended to load into memory and start the execution of one single program that was intended to be executed.

Soon after, by the end of the 50's the idea of loading a single program capable continuously loading and starting user defined programs created the concept of Operating System, at this point all computers were mainframes, a large computer controlled by terminals that had only command line interface (CLI).



OS History

With personal computers, the operating systems became a lot more user-friendly, so the idea of only command-line interface (CLI) was too burdensome for reaching the intended broad audience.



While Microsoft based its whole company on Disk Operating System (DOS), academia converged to a general unifying idea of OS, the Unix System. DOS became Windows when graphics were added. Unix became Linux, FreeBSD, then MacOS, MacOS X, ...



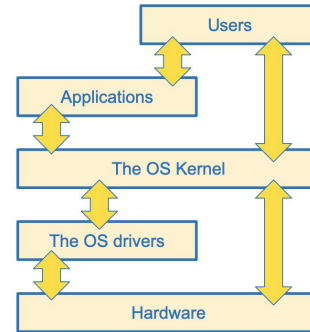
OS Basics Concepts

An OS, including drivers, manages the peripherals and internal units of the machine. The peripherals can be roughly divided into:

- input devices, like a keyboard, a mouse, a trackpad, a camera, etc.
- output devices, like a display, a printer, speakers, etc.
- bi-directional peripheral as the external memory (e.g., disk, flash drive), network connectors, etc.

The internal units comprises:

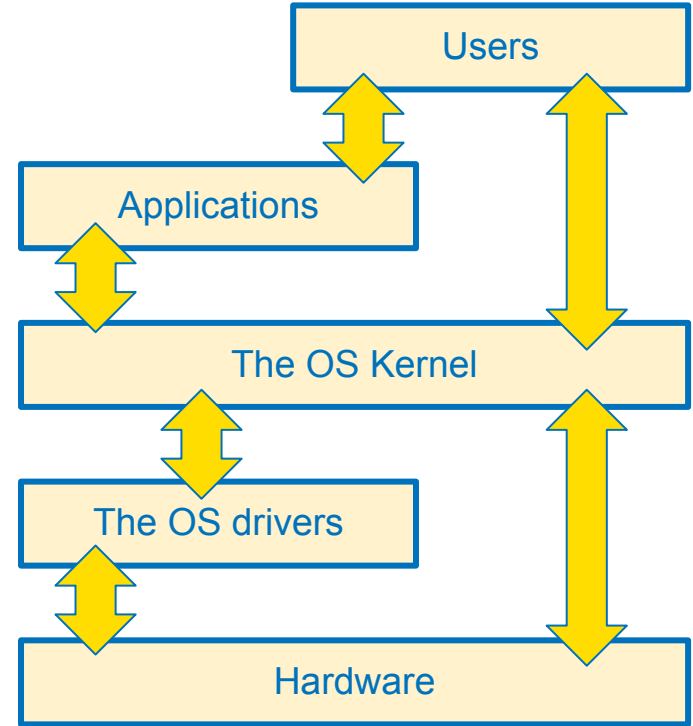
- processing unit management (CPUs)
 - threads (mono and multi-core);
- associated memory hierarchy:
 - memory virtualization;
 - file system.



OS Organization

An OS is generally composed by:

- the kernel is responsible for:
 - the CPUs (cores) scheduling;
 - the memory hierarchy;
 - the file system;
 - calls to hardware drivers;
- the hardware drivers are responsible to interface with peripherals, and are often developed by the peripheral manufacturer (default options come with the OS).



Command Line Interface

The access to the OS is frequently granted by CLI.

The use of Command Line Interface (CLI) may help you to automate tedious tasks, and allow you to bypass some restrictions imposed by the OS Graphical User Interface (GUI).

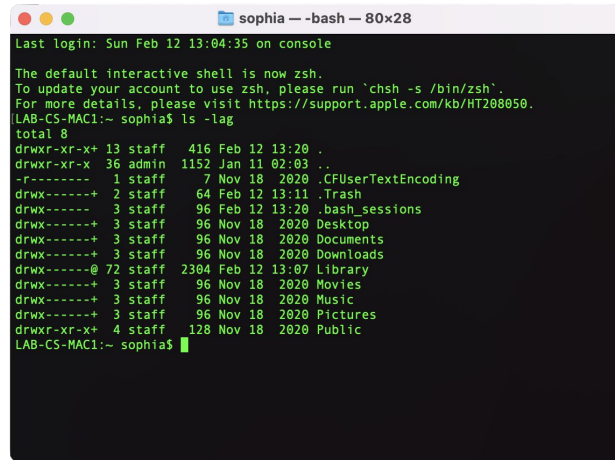
To be able to use the CLI you need to know the script language used by your CLI, and even inside your high-level language you can use such script commands:

- sh scripts for Linux/macOS
- sh scripts for Windows
- System calls inside languages



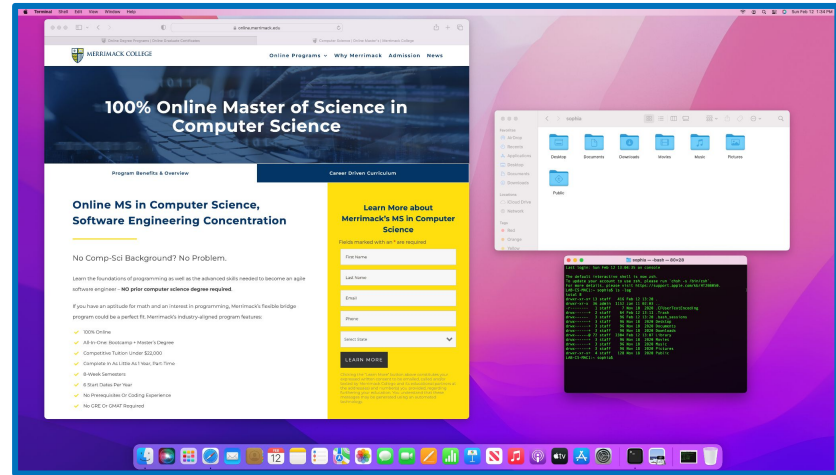
Command-Line Interface

The use a computer directly over the OS can either be done using a Graphical User Interface (GUI) or a Command Line Interface (CLI), which may be burdensome to general users, but still a need for CS professionals.



```
sophia -- -bash -- 80x28
Last login: Sun Feb 12 13:04:35 on console

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
LAB-CS-MAC1:~ sophia$ ls -lag
total 8
drwxr-xr-x+ 13 staff  416 Feb 12 13:20 .
drwxr-xr-x  36 admin 1152 Jan 11 02:03 ..
-r-----+  1 staff   7 Nov 18 2020 .CFUserTextEncoding
drwx-----+ 2 staff  64 Feb 12 13:11 .Trash
drwx-----+ 3 staff  96 Feb 12 13:20 .bash_sessions
drwx-----+ 3 staff  96 Nov 18 2020 Desktop
drwx-----+ 3 staff  96 Nov 18 2020 Documents
drwx-----+ 3 staff  96 Nov 18 2020 Downloads
drwx-----@ 72 staff 2304 Feb 12 13:07 Library
drwx-----+ 3 staff  96 Nov 18 2020 Movies
drwx-----+ 3 staff  96 Nov 18 2020 Music
drwx-----+ 3 staff  96 Nov 18 2020 Pictures
drwxr-xr-x+ 4 staff 128 Nov 18 2020 Public
LAB-CS-MAC1:~ sophia$
```



MERRIMACK COLLEGE

Video: [Command-Line Interface](#).

Command-Line Interface

While, most users use the GUI (it is what you are doing now when you double clicked your browser), the access through CLI is often faster for systematic actions, and sometimes the only option to remote access.

The access to the file system, for example can be done with the CLI or the GUI.

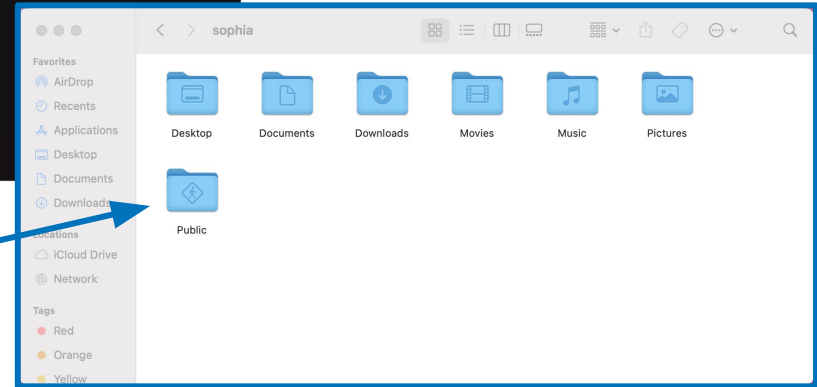
```
sophia -- -bash -- 80x28
Last login: Sun Feb 12 13:04:35 on console

The default interactive shell is now zsh.
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit https://support.apple.com/kb/HT208050.
LAB-CS-MAC1:~ sophia$ ls -lag
total 8
drwxr-xr-x+ 13 staff  416 Feb 12 13:20 .
drwxr-xr-x+ 36 admin 1152 Jan 11 02:03 ..
-r-----+ 1 staff   7 Nov 18 2020 .CFUserTextEncoding
drwx-----+ 2 staff  64 Feb 12 13:11 .Trash
drwx-----+ 3 staff  96 Feb 12 13:20 .bash_sessions
drwx-----+ 3 staff  96 Nov 18 2020 Desktop
drwx-----+ 3 staff  96 Nov 18 2020 Documents
drwx-----+ 3 staff  96 Nov 18 2020 Downloads
drwx-----@ 72 staff 2304 Feb 12 13:07 Library
drwx-----+ 3 staff  96 Nov 18 2020 Movies
drwx-----+ 3 staff  96 Nov 18 2020 Music
drwx-----+ 3 staff  96 Nov 18 2020 Pictures
drwxr-xr-x+ 4 staff 128 Nov 18 2020 Public
LAB-CS-MAC1:~ sophia$
```

`ls -lag`

Invisible folder

Visible folder



MERRIMACK COLLEGE

Text: Command Line Interface.

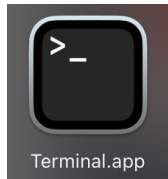
Accessing the CLI - using a shell

In a Unix based OS you can access the CLI calling a **terminal** native program.

In MacOS you can go to Applications, Utilities folder, then the **terminal** app.



In a Windows OS you can access the the **Command Line Prompt** (old systems) or install **PowerShell** (since 2022 accessible as an option of terminal app to host command-line interface).



MERRIMACK COLLEGE

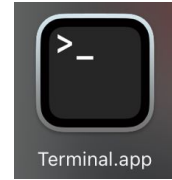
Resource: [Installing PowerShell on Windows.](#)

sh scripts for Linux/MacOS

```
1  #!/bin/bash
2  echo "Hello World, I'm bash"
```

For a Unix inspired OS, and MacOS is one of those, it is usual to have terminal using the script "language" Unix Shell (**sh**), an interpreted language capable to have access the OS primitive services, specially the file system, but actually all OS available resources.

```
1  #!/bin/bash
2  cd
3  pwd
4  echo "Enter directory name"
5  read ndir
6  if [ -d "$ndir" ]
7  then
8  echo "Directory exist"
9  else
10 `mkdir $ndir`
11 echo "Directory created"
12 fi
```

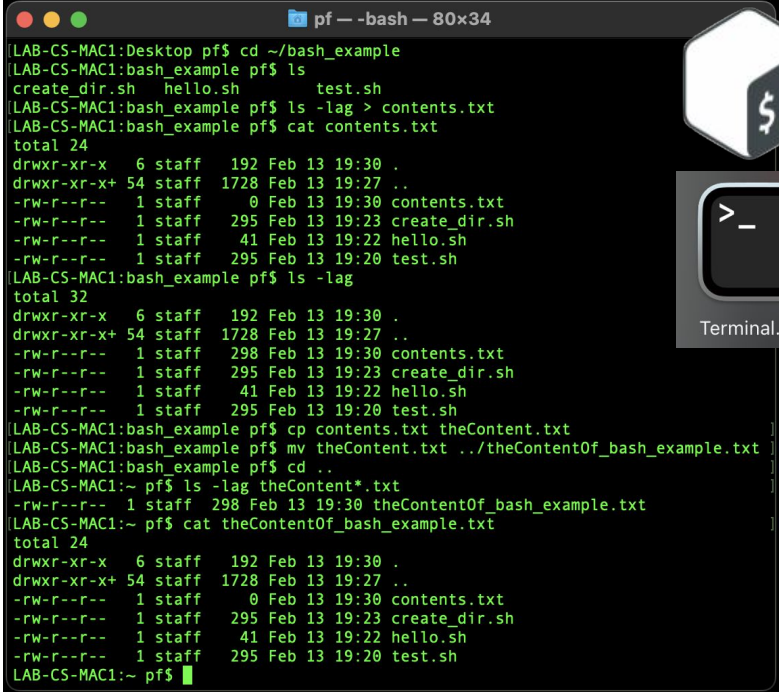


For MacOS, it is usual to have as default a version of **sh** called **bash**, a **sh** written by Brian Fox, making a geek joke about *Bourne Shell*, as *Bourne Again Shell*.



bash (Linux/MacOS) - basic commands

- `cd ~/bash_example`
 - go to the folder *bash_example* at your home folder
- `ls`
 - simple display of the current folder content
- `ls -lag > contents.txt`
 - detailed display of the current folder content and saving it in a file *contents.txt*



A screenshot of a macOS Terminal window titled "pf - bash - 80x34". The terminal shows a series of commands and their outputs. The user navigates to the `~/bash_example` directory and lists files with `ls`. Then, they use `ls -lag > contents.txt` to save a detailed listing to a file. Finally, they copy the file to the parent directory and move it to a new directory named `theContentOf_bash_example.txt`. The output of `ls -lag` shows file permissions, owner, group, size, date, and filename for each item in the directory.

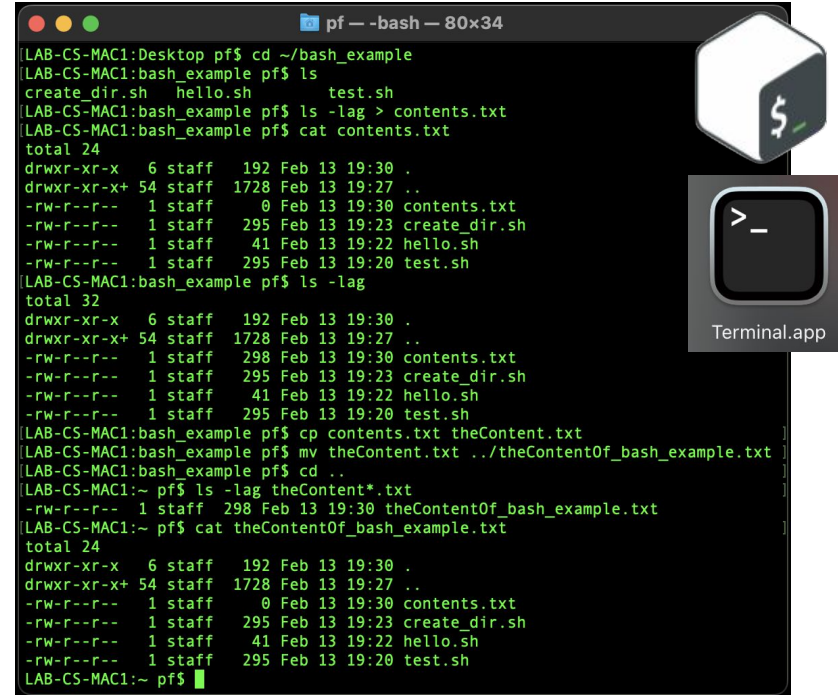
```
LAB-CS-MAC1:Desktop pf$ cd ~/bash_example
LAB-CS-MAC1:bash_example pf$ ls
create_dir.sh  hello.sh      test.sh
LAB-CS-MAC1:bash_example pf$ ls -lag > contents.txt
LAB-CS-MAC1:bash_example pf$ cat contents.txt
total 24
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff   0 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:bash_example pf$ ls -lag
total 32
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff  298 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:bash_example pf$ cp contents.txt theContent.txt
LAB-CS-MAC1:bash_example pf$ mv theContent.txt ../theContentOf_bash_example.txt
LAB-CS-MAC1:~ pf$ ls -lag theContent*.txt
-rw-r--r--  1 staff  298 Feb 13 19:30 theContentOf_bash_example.txt
LAB-CS-MAC1:~ pf$ cat theContentOf_bash_example.txt
total 24
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff   0 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:~ pf$
```

1



bash (Linux/MacOS) - basic commands

- `cat contents.txt`
 - displays the contents of file *contents.txt*
- `ls -lag`
 - detailed display of the current folder content
- `cp contents.txt theContent.txt`
 - copy the file *contents.txt* to a new file *theContent.txt*



A screenshot of a macOS Terminal window titled "pf - bash - 80x34". The window shows a series of commands and their outputs in a dark-themed interface. The commands executed are: `cd ~/bash_example`, `ls`, `ls -lag > contents.txt`, `cat contents.txt`, `ls -lag`, `cp contents.txt theContent.txt`, `mv theContent.txt ../theContentOf_bash_example.txt`, `ls -lag theContent*.txt`, and `cat theContentOf_bash_example.txt`. The output of `ls -lag` shows a directory listing with permissions, owner, group, size, date, and filename. The output of `cat` shows the contents of the files. The window also features a dock icon for "Terminal.app" on the right side.

```
LAB-CS-MAC1:Desktop pf$ cd ~/bash_example
LAB-CS-MAC1:bash_example pf$ ls
create_dir.sh  hello.sh      test.sh
LAB-CS-MAC1:bash_example pf$ ls -lag > contents.txt
LAB-CS-MAC1:bash_example pf$ cat contents.txt
total 24
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff   0 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:bash_example pf$ ls -lag
total 32
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff  298 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:bash_example pf$ cp contents.txt theContent.txt
LAB-CS-MAC1:bash_example pf$ mv theContent.txt ../theContentOf_bash_example.txt
LAB-CS-MAC1:~ pf$ ls -lag theContent*.txt
-rw-r--r--  1 staff  298 Feb 13 19:30 theContentOf_bash_example.txt
LAB-CS-MAC1:~ pf$ cat theContentOf_bash_example.txt
total 24
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff   0 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:~ pf$
```

2



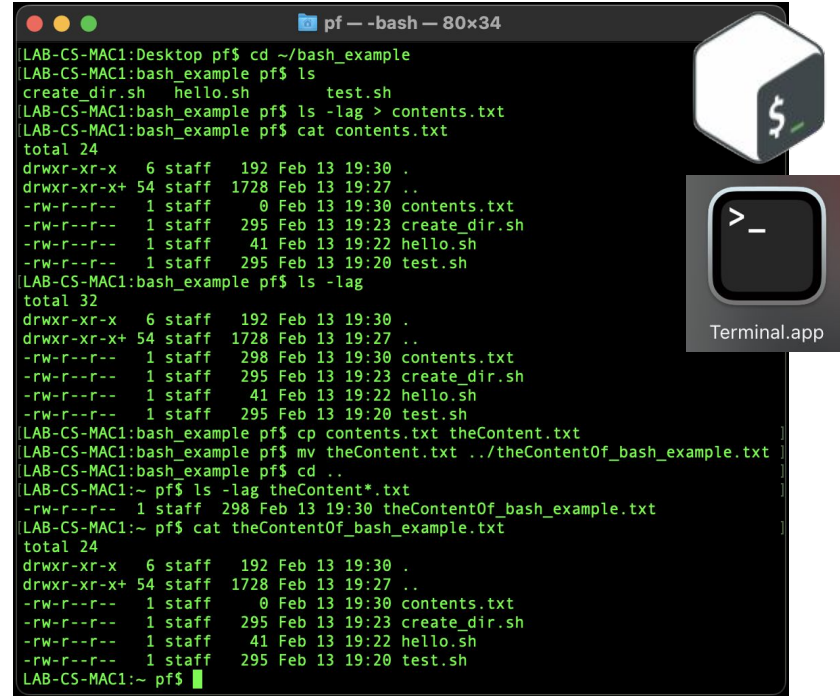
MERRIMACK COLLEGE

Video: [Intro to the Command Line - BASH Basics](#).

bash (Linux/MacOS) - basic commands

- `mv theContent.txt`
`../theContentOf_bash_example.txt`
 - move the file *theContent.txt* to the upper folder naming it *theContentOf_bash_example.txt*
- `cd ..`
 - go to the upper folder
- `ls -lag theContent*.txt`
 - detailed display of the content of the current folder that match the mask *theContent*.txt*

3



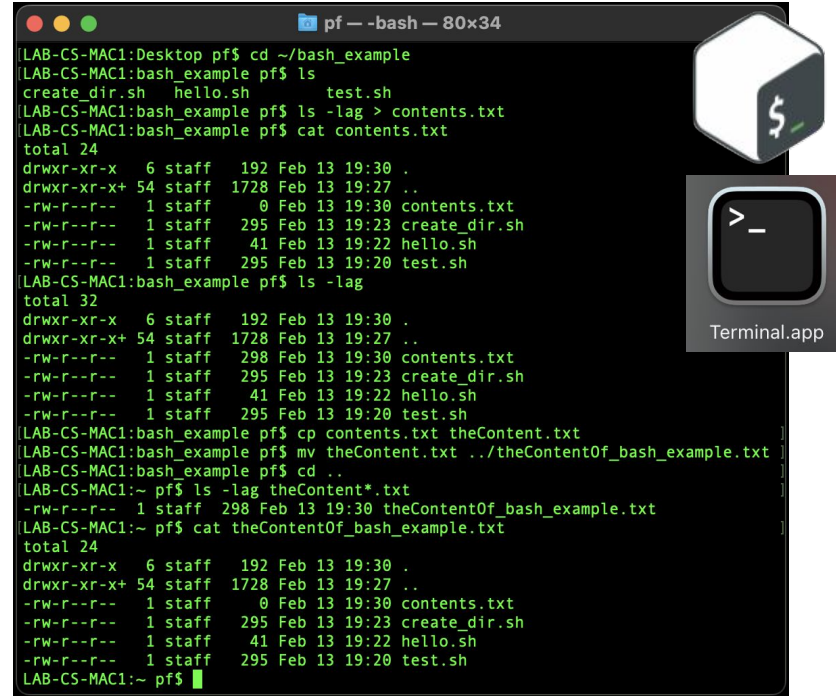
A screenshot of a macOS Terminal window titled "pf - bash - 80x34". The window shows a series of commands and their outputs. The user starts in the directory ~/bash_example. They run 'ls' showing files create_dir.sh, hello.sh, and test.sh. Then they run 'ls -lag > contents.txt' to save the directory listing. Next, they run 'cat contents.txt' which displays a detailed 'ls -lag' output for the current directory. Then they run 'ls -lag' again, showing the same output. Finally, they run 'cp contents.txt theContent.txt' and 'mv theContent.txt ../theContentOf_bash_example.txt'. The prompt then changes to the parent directory (~), and they run 'ls -lag theContent*.txt' which shows the file 'theContentOf_bash_example.txt' with its detailed permissions and metadata. The prompt returns to the parent directory (~).

```
LAB-CS-MAC1:Desktop pf$ cd ~/bash_example
LAB-CS-MAC1:bash_example pf$ ls
create_dir.sh  hello.sh  test.sh
LAB-CS-MAC1:bash_example pf$ ls -lag > contents.txt
LAB-CS-MAC1:bash_example pf$ cat contents.txt
total 24
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff   0 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:bash_example pf$ ls -lag
total 32
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff  298 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:bash_example pf$ cp contents.txt theContent.txt
LAB-CS-MAC1:bash_example pf$ mv theContent.txt ../theContentOf_bash_example.txt
LAB-CS-MAC1:~ pf$ ls -lag theContent*.txt
-rw-r--r--  1 staff  298 Feb 13 19:30 theContentOf_bash_example.txt
LAB-CS-MAC1:~ pf$ cat theContentOf_bash_example.txt
total 24
drwxr-xr-x  6 staff  192 Feb 13 19:30 .
drwxr-xr-x+ 54 staff 1728 Feb 13 19:27 ..
-rw-r--r--  1 staff   0 Feb 13 19:30 contents.txt
-rw-r--r--  1 staff  295 Feb 13 19:23 create_dir.sh
-rw-r--r--  1 staff  41 Feb 13 19:22 hello.sh
-rw-r--r--  1 staff  295 Feb 13 19:20 test.sh
LAB-CS-MAC1:~ pf$
```



bash (Linux/MacOS) - basic commands

- `cat theContentOf_bash_example.txt`
 - displays the contents of the file *theContentOf_bash_example.txt*
- Now, or after the live session finishes, go and try these commands by yourselves
- Try also typing:
 - `man ls`
 - It should display the manual page for the command `ls` (scroll with arrows, exit with `q`)



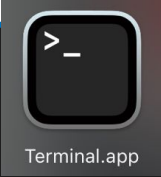
A screenshot of a macOS Terminal window titled "pf - bash - 80x34". The window shows a series of commands and their outputs. The commands include `cd ~/bash_example`, `ls`, `create_dir.sh hello.sh`, `test.sh`, `ls -lag > contents.txt`, `cat contents.txt`, `ls -lag`, `cp contents.txt theContent.txt`, `mv theContent.txt ../theContentOf_bash_example.txt`, `ls -lag theContent*.txt`, and `cat theContentOf_bash_example.txt`. The output of `ls -lag` shows a directory listing with permissions, owner, group, size, date, and filename. The output of `cat theContentOf_bash_example.txt` shows the contents of the file, which are the same as the commands entered in the terminal.

4



bash (Linux/MacOS) - basic commands

- to go a folder: **cd** <destination>
- to display where you are: **pwd**
- to output something: **echo** "<text>"
- to receive an input: **read** <variable>
- to decide: **if** [<condition>] **then** <commands> **else** <commands> **fi**
- to check if a file is an existing directory: **-d** "<name>"
- to refer to the content of a variable: **\$**<variable>
- to create a directory: **`mkdir** <name>`



```
1  #!/bin/bash
2  cd
3  pwd
4  echo "Enter directory name"
5  read ndir
6  if [ -d "$ndir" ]
7  then
8  echo "Directory exist"
9  else
10 `mkdir $ndir`
11 echo "Directory created"
12 fi
```



sh scripts for Windows

The origin of the scripts for Windows is the MS-DOS, but it is undeniable that the Unix-based script is a more suitable approach. Around 2002, Microsoft attempt to create a CLI closer to Unix, but it remained impossible based on fundamental differences between Windows and Unix-like OSs.

```
1 # PowerShell
2 Write-Host 'Hello World, I'm PowerShell'
```



PowerShell

```
1 # PowerShell
2 $name = Read-Host -Prompt 'Enter directory name'
3 if (Test-Path $name) {
4     Write-Host "Directory exist"
5 }
6 else {
7     New-Item $name -ItemType Directory
8     Write-Host "Directory created"
9 }
```



MERRIMACK COLLEGE

Wikipedia: [PowerShell](https://en.wikipedia.org/wiki/PowerShell).

PowerShell (Windows) - basic commands

PowerShell is both a command-line shell and a scripting language.

While Perl was an attempt to offer a high-level language to be as direct as Unix-like shells, PowerShell aims the same goal to be a language to automate system administration.

The approach of a higher-level abstraction language (using *Cmdlets*) and several distinct commands and parameters sets PowerShell apart from Unix sh.



PowerShell

Many PowerShell *command aliases* mimic the Unix-like ones as:

- **cat**, cd, **clear**, cp, **echo**, ls, **mv**, pwd, **rm**

However, it also offer legacy options as the old ones from MS command-line prompt:

- **gc**, chdir, **clr**, copy, **echo**, dir, **move**, gl, **del**



PowerShell (Windows) - basic commands

- `$<variable> Read-Host -Prompt <msg>`
 - to get input towards a variable
- `if ( <condition> ) { <commands> }`
`else { <commands> }`
 - to decide
- `Write-Host "<msg>"`
 - to output text
- `New-Item <name>`
`-ItemType <Directory`
`or File>`
 - to create an item

If you are curious about old MS Command Prompt [here](#) is a video!



PowerShell

```
1  # PowerShell
2  $name = Read-Host -Prompt 'Enter directory name'
3  if (Test-Path $name) {
4      Write-Host "Directory exist"
5  }
6  else {
7      New-Item $name -ItemType Directory
8      Write-Host "Directory created"
9  }
```



MERRIMACK COLLEGE

Text: [PowerShell - Tutorial.](#)

System calls within high-level languages

- In Python:

- `import os`
- `os.system("<command and parameters>")`



All
languages
offer
system
calls

- In C++:

- `#<stdlib.h> or #<cstdlib>`
- `int system("<command and parameters>")`
 - it returns a number (0 for execution Ok)



- In Go:

- `import "os/exec"`
- `var <variable> = exec.Command(<command name>, arg1, arg2)`
 - it return to the variable the output of the command



This Week's tasks

- Post discussion
- Programming project #5

Tasks

- Comment on **likely you believe you will use a shell script language** and **why**.
- Create a Python program **that creates a folder** and **saves a file** with 100 random numbers **inside this folder**.



Post Discussion - Week 5 - In-class exercises

Comment on likely you believe you will use a shell script language and why.

- High-level programming languages are easier to organize your thoughts, understand the code modules, and you are now use to it, but some tasks are more easily done using a script shell language. So, can you imagine yourself using such kind of language?
 - Feel free to give personal reasons, but relate those with technical characteristics.

Your task:

- Post your discussion in the message board until this Saturday;
- Reply posts of your colleagues in the message board until next Wednesday.



MERRIMACK COLLEGE

This task counts towards the In-Class Exercises grade.

Progr. Project #5 - Week 5 - P#5

Create a Python program that creates folder and saves a file with 100 random numbers inside this folder.

- Your program should ask the user the name of the folder to be created
 - if the folder exists, the folder has to be removed and a new one created;
- Once created, your program should generate 100 random number from 0 to 1000 and save them in a file named "numbers100.txt" inside the created folder.

The program must perform system calls to your operating system in order to check if the folder exists, maybe delete it, and to create the folder.

Feel free to research on the Internet ways to check a folder existence in your own OS.



Progr. Project #5 - Week 5 - P#5

- You must submit Python code in a single file.
- If you need to add any explanation or messages to me, please make it in the code remarks.
- The quality of your code remarks is not relevant for the evaluation, but make sure you write your name in the remarks, as well as to which system (version included) your code was tested on.
 - It is not necessary, but a good practice to properly document your code.

Your task:

- Save your file and submit it in the appropriate Canvas until Next Wednesday.

This assignment counts towards
the Coding Projects grade.



MERRIMACK COLLEGE

” Week 5 of CSC 6303

- **Post in the discussions until Saturday, reply until next Wednesday**
- **Do the coding project until next Wednesday**

Next Week - File system basic concepts and styles



MERRIMACK COLLEGE